

CSCI 220 - Data Structures & Algorithms

Module 0 Quiz

Spring 2024

Problem 1:

Circle the answer which best matches the term or concept:

(a) pointer

- A. Another name for the “dot” operator used to access member variables and functions.
- B. In source code, a comment which directs the reader to another source file.
- C. **Correct** A variable storing the memory address of a variable, array, object, etc.
- D. A call to a function within a function of the same name.

(b) operators that dereference a pointer

- A. **Correct** *, ->, []
- B. &, *, ::
- C. ->, [], &
- D. *, delete, ->

(c) free store (or heap)

- A. Memory region where local variables and function arguments reside.
- B. **Correct** Memory region where dynamically allocated memory resides.
- C. Memory region where global variables and static member variables reside.
- D. Memory region where program code resides.

(d) big 3

- A. Default constructor, assignment operator, deallocation operator.
- B. **Correct** Copy constructor, assignment operator, destructor.
- C. Addition/concatenation operator, equality operator, destructor.
- D. Copy constructor, equality operator, deallocation operator.

(e) push

- A. **Correct** Add an element to the top of a stack.
- B. Remove the top element of a stack.
- C. Move the second element of a queue to the front.
- D. Clear all elements from a stack.

(f) generic programming

- A. Non-brand name code in plain packaging.
- B. When one class takes on the member variables and methods of another, parent class.
- C. The universally agreed on C++ style guidelines used by all developers.
- D. **Correct** Writing program code for an algorithm or data structure such that the code can be applied to different types as needed.

Problem 2:

The code below compiles and runs, but exits with a “Segmentation fault” error before completing.

```
1  int main() {
2      int** matrix = new int*[1024];
3      for (int i = 0; i < 1024; i++) {
4          matrix[i] = new int[100];
5          for (int j = 0; j < 100; j++) {
6              matrix[i][j] = rand();
7          }
8      }
9
10     delete[] matrix;
11
12     double* averages = new double[1024];
13     for (int i = 0; i < 1024; i++) {
14         averages[i] = 0.0;
15         for (int j = 0; j < 100; j++) {
16             averages[i] += matrix[i][j];
17         }
18         averages[i] = averages[i] / 100.0;
19     }
20
21     cout << "Last average is: " << averages[1023] << endl;
22     delete[] averages;
23 }
```

This code has not one, but two memory-related errors. Identify and explain at least one of the errors. If you can identify and explain both, even better!

All of the errors begin with line 10 when we deallocate `matrix`. The first error, which is perhaps less severe in that it doesn't cause the immediate crash of our program, is a memory leak. By deallocating `matrix` without first deallocating all of the sub-arrays allocated in line 4, we effectively lose the pointers to all of those sub-arrays, causing a **memory leak**.

The second error leads to the program crashing on line 16. The cause of the crash is our use of the **dangling pointer**, `matrix`. We previously deallocated the array pointed to by `matrix`, and when we allocate memory for `averages`, we may well have gotten a pointer to the *same* region of memory that `matrix` was formerly using; and now we are overwriting that memory. We actually initialize that memory to 0 in line 14 - so `matrix[i]` in 16 is very likely to now be a null pointer, which we try to dereference further (with `[j]`).

Problem 3:

For each snippet of code below, write what the code prints to standard output:

(a)

```
1  int main() {
2      queue<string> q;
3      stack<string> s;
4      vector<string> elements = {"orange", "banana", "grape", "apple"};
5      for(int i = 0; i < elements.size(); i++) {
6          q.push(elements[i]);
7          s.push(elements[i]);
8      }
9      q.pop();
10     s.pop();
11     q.push("mango");
12     s.push("mango");
13     q.push("papaya");
14     s.push("papaya");
15     q.pop();
16     s.pop();
17     while(!q.empty()) {
18         cout << q.front() << " ";
19         q.pop();
20     }
21     cout << endl;
22     while(!s.empty()) {
23         cout << s.top() << " ";
24         s.pop();
25     }
26     cout << endl;
27     return 0;
28 }
```

```
grape apple mango papaya
mango grape banana orange
```

(b)

```
1 int main() {
2     float arr[5] = {10.5, 12.0, 18.5, 72.5, 0.9};
3     float *ptr1 = arr;
4     float *ptr2 = ptr1 + 3;
5     ptr1++;
6
7     cout << *ptr1 << " ";
8     cout << *ptr2 << " ";
9     cout << ptr2 - ptr1 << endl;
10    return 0;
11 }
```

A trace of main

ptr1	ptr2	commentary
&arr[0]	??	@main:3, the pointer is assigned the location of <code>arr</code> , which is also the address of <code>arr</code> 's first element.
&arr[0]	&arr[3]	@main:4, the pointer is assigned the location 3 elements forward of <code>arr[0]</code> , which is the location of 72.5 in <code>arr</code> .
&arr[1]	&arr[3]	@main:5, the pointer is incremented forward to point to the next element of <code>arr</code> , in other words <code>&arr[1]</code>
&arr[1]	&arr[3]	@main:7, the value at <code>arr[1]</code> is printed, (12).
&arr[1]	&arr[3]	@main:8, the value at <code>arr[3]</code> is printed, (72.5).
&arr[1]	&arr[3]	@main:9, the value 2 is printed, because there are two elements between array locations <code>ptr2=&arr[3]</code> and <code>ptr1=&arr[1]</code> , ($2 = 3 - 1$)

Another way to think of the last tracing step: C++ respects the “algebra” of pointers.

$$ptr1 + 2 = ptr2 \Rightarrow 2 = ptr2 - ptr1$$

The actual output

12 72.5 2

(c)

```
1  class blue {
2  public:
3      string s;
4      blue(const string& val) { s = val; }
5      blue operator/(blue& b) { return blue(s + b.s); }
6  };
7  ostream& operator<<(ostream& out, blue b) {
8      out << b.s;
9      return out;
10 }
11
12 template <class T>
13 void do_something(T u, T v) {
14     cout << u / v << endl;
15 }
16
17 int main() {
18     blue x("Go");
19     blue y("Orediggers!");
20     do_something(170, 10);
21     do_something(x, y);
22     return 0;
23 }
```

A trace of main

x.s	y.s	commentary
"Go"	??	@main:18, string param constructor on line 4 is called for x
"Go"	"Orediggers!"	@main:19, string param constructor on line 4 is called for y
"Go"	"Orediggers!"	@main:20, do_something(int& u, int& v) called with &u=170, &v=10 at line 14 prints out 17
"Go"	"Orediggers!"	@main:21, do_something(blue& u, blue& v) called with &u=x, &v=y at line 14
"Go"	"Orediggers!"	@do_something<blue,blue>:14 calls blue::operator/ with this=&x and &b=y
"Go"	"Orediggers!"	@blue::operator/:5 calls the blue string param constructor with s="GoOrediggers!" the new object is an "unnamed temporary" and returned to do_something<blue,blue>.
"Go"	"Orediggers!"	@do_something<blue,blue>:14 calls operator<< with &out=cout and the unnamed temporary as b
"Go"	"Orediggers!"	The b parameter of operator<< is constructed from do_something<blue,blue>'s unnamed temporary using the default copy constructor provided by the compiler.
"Go"	"Orediggers!"	Finally, GoOrediggers! is printed by operator<< at line 7.

The actual output

```
17
GoOrediggers!
```

(d)

```
1  class yellow {
2  public:
3      int data;
4      yellow() { data = 1; }
5      yellow(const yellow& y) {
6          data = y.data * 3;
7      }
8      yellow& operator=(const yellow& y) {
9          if (&y == this) return *this;
10         data = y.data - 1;
11         return *this;
12     }
13     ~yellow() { cout << "Hey!" << endl; }
14 };
15
16 int main() {
17     yellow a;
18     yellow b = a;
19     yellow c = b;
20     a = b;
21     cout << a.data << " " << b.data << " " << c.data << endl;
22     c = b = a;
23     cout << a.data << " " << b.data << " " << c.data << endl;
24 }
```

A trace of main

a	b	c	commentary
1	??	??	@main:17, default constructor on line 4
1	3	??	@main:18, copy constructor on line 5
1	3	9	@main:19, copy constructor on line 5
2	3	9	@main:20, assignment op on line 8 with this=&a and y=b
2	3	9	first output of main
2	1	9	@main:22, assignment op on line 8 for the rightmost =, this=&b and y=a
2	1	0	@main:22, assignment op on line 8 for the leftmost =, this=&c and y=b
2	1	0	second output of main
2	1	0	Hey! printed by first of three destructors at line 13
2	1	0	Hey! printed by second of three destructors at line 13
2	1	0	Hey! printed by third of three destructors at line 13

The actual output

```
2 3 9
2 1 0
Hey!
Hey!
Hey!
```